

ML Reading Group: The Final Meeting of the Year

Comprehensive Meeting Notes

May 10, 2026

Contents

1	Neural Thickets — Prastik	2
1.1	Overview	2
1.2	Key Ideas and Technical Details	2
1.3	Experimental Results and Evidence	3
1.4	Discussion and Q&A	4
1.5	Key Takeaways and Open Questions	4
2	Flow Q-Learning (FQL) — Mark	5
2.1	Overview	5
2.2	Key Ideas and Technical Details	5
2.3	Experimental Results and Evidence	7
2.4	Discussion and Q&A	7
2.5	Key Takeaways and Open Questions	8
3	METRA — Deepinder	9
3.1	Overview	9
3.2	Key Ideas and Technical Details	9
3.3	Experimental Results and Evidence	12
3.4	Discussion and Q&A	12
3.5	Key Takeaways and Open Questions	13
4	Randomized Numerical Linear Algebra for ML — Surya	15
4.1	Overview	15
4.2	Key Ideas and Technical Details	15
4.3	Experimental Results and Evidence	17
4.4	Discussion and Q&A	18
4.5	Key Takeaways and Open Questions	18
5	The Muon Optimizer — Dhruv	19
5.1	Overview	19
5.2	Key Ideas and Technical Details	19
5.3	Experimental Results and Evidence	21
5.4	Discussion and Q&A	21
5.5	Key Takeaways and Open Questions	22

Overview

Neural Thickets is a paper from Phillip Isola’s group (published on arXiv in March) that proposes a fundamentally different approach to post-training language models. Rather than treating a pre-trained parameter vector as a fixed starting point for iterative, gradient-based fine-tuning, the paper argues that a sufficiently large pre-trained model induces a “thicket” in parameter space: a dense, high-dimensional neighborhood around the pre-trained weights in which many purely random perturbations already correspond to improved, task-specialized models. The core contribution is an algorithm called **RandOpt** which samples n Gaussian perturbations of the pre-trained weight vector in parallel, evaluates each on a held-out set, retains the top- k , and combines them via majority vote at inference time. The approach is non-gradient-based and fully parallelizable, distinguishing it from sequential algorithms such as PPO, GRPO, and evolutionary strategies.

Key Ideas and Technical Details

Motivation: From Needle to Thicket

In a low-dimensional or randomly initialized setting, sampling a random perturbation of a parameter vector is almost certainly unhelpful — the “needle in a haystack” regime. The central claim of the paper is that this intuition reverses for very large, pre-trained models: the neighborhood around the pre-trained weights becomes a “thicket,” where many random perturbations are beneficial for a given downstream task.

The Core Algorithm

The algorithm is described in two phases.

Setup (sampling): Given a pre-trained parameter vector θ , sample n perturbed vectors:

$$\theta_i = \theta + \sigma \cdot \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, I), \quad i = 1, \dots, n.$$

Here σ is a fixed scalar (set to approximately 0.05 in experiments; not learned). The perturbations are isotropic — the same variance in every parameter dimension.

Selection (top- k): Each perturbed model θ_i is evaluated on a small task-specific validation set. The top- k models by accuracy (or loss) are retained.

Inference (ensembling): For a given query, all k retained models are run in parallel and their outputs are aggregated, e.g., by majority vote. For classification tasks (reaction type prediction, math problem type, etc.) this is straightforward.

The entire procedure is non-iterative and non-gradient-based, making it embarrassingly parallel. The presenter summarized: “You don’t even have to know how the model actually does [internally]. You’re not really touching the model.”

Key Properties the Paper Aims to Establish

The paper attempts to justify the algorithm through four empirical claims:

1. **Density:** The neighborhood around θ is dense in good perturbations. Formally, they define a margin-of-improvement metric $\delta(m)$:

$$\delta(m) = \Pr_{\varepsilon \sim \mathcal{N}(0, I)} [\text{Acc}(\theta + \sigma \varepsilon) \geq \text{Acc}(\theta) + m],$$

where $m \geq 0$ is a margin parameter. Setting $m = 0$ asks whether any perturbation helps at all; larger m requires a larger improvement.

2. **Diversity:** Different perturbations specialize to different tasks. The paper quantifies this via the off-diagonal elements of a cross-task correlation matrix: if perturbation j is good at math, it should perform worse at chemistry relative to a perturbation that is good at chemistry. Diversity is measured as $1 - \sum_{j \neq k} |\rho_{jk}|$ for this correlation matrix ρ .
3. **Scaling with model size:** Both density and diversity are claimed to increase with the number of model parameters. Experiments use the Qwen family, ranging from approximately 0.5B to 32B parameters.
4. **Ensemble utility:** Together, density and diversity justify the RandOpt approach.

Theoretical Hypotheses (Presenter’s Additions)

The presenter emphasized that the paper provides no theoretical explanation for why the thicket phenomenon occurs. Prastik offered two hypotheses:

Hypothesis 1 — Overparameterization and degenerate solution manifolds: In a vastly overparameterized system (e.g., 32B parameters is sufficient to represent English), the set of solutions for any given task is not a single point but a high-dimensional manifold. A Gaussian ball centered at θ will, with high probability, intersect this manifold. An important geometric intuition: in very high dimensions, Gaussian mass concentrates in a thin spherical shell rather than uniformly in the ball. This means that rather than sampling along radii, one effectively samples directions — and different directions may intersect different task-specialized manifolds.

Hypothesis 2: [Mentioned briefly but not fully developed in the available time.]

Tasks and Benchmarks

The paper evaluates on approximately ten tasks across four domains: mathematics (including arithmetic fill-in, competition-style problems), chemistry (reaction mechanism classification, e.g., SN1 vs. E2), computer science (Python coding tasks), and creative writing (story generation). Results are presented for the Qwen model family.

Experimental Results and Evidence

The paper shows that for large enough models, a majority of sampled perturbations outperform the original θ on a given task (by at least $m = 0$ margin), though the magnitude of improvement is often modest. Appendix Figure 12 (referenced during the talk) shows overlapping distributions of perturbed vs. baseline performance, with the perturbed distribution shifted slightly upward. The presenter noted: “What they were able to show is that a majority do better, but by a very marginal amount.”

Diversity results show anti-correlation between task performance across different sampled perturbations, supporting the specialization hypothesis. However, the presenter questioned not directly testing whether improvement on math measurably degrades performance on chemistry (only correlation is shown, not causal subtraction).

Scaling results with the Qwen family (4–6 model sizes from ≈ 0.5 B to 32B) show a trend of increasing density and diversity with model size, though the presenter felt the model family was too small to be fully convincing.

Discussion and Q&A

- **Audience (Reese) asked** what a “neural thicket” is. The presenter explained: the term refers to the dense region of beneficial perturbations around the pre-trained parameter vector.
- **Audience asked** whether this is similar to evolutionary strategies. The presenter distinguished them: evolutionary strategies are also perturbation-based but are sequential (iterative), whereas this approach is purely parallel.
- **Audience asked** whether σ is learned. The presenter confirmed it is not learned — it is a fixed hyperparameter (set to ≈ 0.05), and learning it would reintroduce the iterative/sequential cost the paper seeks to avoid.
- **Audience asked** whether the sampling is applied to all weights or only specific layers. The presenter indicated the paper samples the entire weight vector and does not freeze any layers, though this was described as a potential area for improvement.
- **Audience observed** that the isotropic assumption is strong. The presenter agreed: “It’s a very rigid space they’re working in.” Anisotropic or layer-wise noise was not explored.
- **Audience asked** whether a good direction for one task could be fixed and scaled up (analogous to iterating). The presenter noted this would reintroduce sequential steps: “You do one perturbation, you see this dimension is good, you want to perturb it more — and the goal is to never touch that.”
- **Audience asked** whether perturbations transfer across model sizes. The presenter noted this is not tested and would require normalizing σ in a principled way.
- **Audience asked** about building a graph over tasks and using it to set σ adaptively (e.g., smaller σ for nearby tasks). The presenter found this a compelling extension: “Maybe the math ones are close to each other, so you can perturb with a smaller sigma.”
- A key area of improvement is the absence of any principled explanation for the observed phenomenon. “They have no methods for why it should even happen... how can a random perturbation actually do something?”

Key Takeaways and Open Questions

1. Large pre-trained models appear to exhibit a “thicket” property: many random isotropic perturbations yield measurable task-specific improvement, suggesting the solution manifold for a fine-tuning task is high-dimensional and likely to be intersected by a Gaussian ball.
2. The algorithm (RandOpt) is compelling for its parallelizability and zero-gradient cost, but achieves only modest accuracy gains over baselines, and the paper does not achieve state-of-the-art results.
3. A key missing theoretical contribution is any principled explanation of why the thicket exists. The overparameterization/solution-manifold argument is compelling but is the presenter’s own hypothesis, not from the paper.
4. Open questions: (a) Does improvement on one task actively harm another? (b) Can anisotropic or layer-wise noise improve results? (c) Can the method scale to different architectures or models beyond 32B? (d) Are there safety regressions from random perturbations? (e) Can σ be chosen adaptively based on task similarity?
5. The approach may be most valuable not as a standalone fine-tuning method but as a diagnostic tool or as a component of a larger pipeline (e.g., distillation from the k specialized models into a single model).

Overview

Flow Q-Learning (FQL) is a method for offline reinforcement learning that addresses a fundamental challenge: behavioral cloning with standard unimodal policies (e.g., Gaussian) fails to capture the multimodal action distributions common in human demonstration data. FQL combines three ingredients — a **flow-based behavior cloning policy**, a **Q-function** trained on offline data, and a **one-step distilled policy** — to produce an expressive yet computationally efficient policy that is nudged toward high-reward regions of the action space. The method is notable for its simplicity and strong empirical performance relative to baselines, while also being the subject of a theoretical critique showing that FQL’s fixed point is not the true reward-tilted distribution.

Key Ideas and Technical Details

Offline Reinforcement Learning Setting

In offline RL, the agent has access to a fixed dataset $\mathcal{D} = \{(s, a, r, s')\}$ of transitions collected by some behavior policy (e.g., human teleoperations). The agent cannot interact with the environment during training. The challenge is to learn a policy that improves upon the behavior policy without querying the environment, while avoiding extrapolation to out-of-distribution state-action pairs where the learned Q-function may be unreliable.

A popular approach is to constrain the learned policy to remain close to the behavior policy (policy constraint methods), rather than being overly pessimistic (e.g., CQL).

Why Flow Models?

A key motivation is that human demonstration data is often **multimodal**: for a given state, a human might move left or right, and a Gaussian policy (mean + standard deviation) would average over these modes, producing an action that goes straight into an obstacle. Flow models (equivalently, continuous normalizing flows or flow-matching models) can represent arbitrary complex distributions over actions.

The flow model is trained using the standard flow-matching objective. Let $x_0 \sim \mathcal{N}(0, I)$ be noise, $x_1 = a$ be the target action, and $x_t = (1 - t)x_0 + tx_1$ be the interpolation. The behavior cloning flow policy π_ϕ is trained to predict the velocity field:

$$\mathcal{L}_{\text{BC}}(\phi) = \mathbb{E}_{t, (s, a) \sim \mathcal{D}, x_0 \sim \mathcal{N}} \left[\|v_\phi(x_t, t, s) - (x_1 - x_0)\|^2 \right].$$

The Four Steps of FQL

1. **Train behavior cloning (BC) policy π_ϕ** : A flow model trained to mimic the data distribution $\pi_\beta(a | s)$ using the objective above.
2. **Train Q-function Q_ψ** : Standard Bellman backup on the offline dataset:

$$\mathcal{L}_Q(\psi) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[(Q_\psi(s, a) - r - \gamma Q_\psi(s', a'))^2 \right],$$

where $a' \sim \pi_\phi(\cdot | s')$ (actions sampled from the BC policy for the target).

3. **Distill BC policy into one-step policy π_ω** : Rather than running the flow ODE for T steps at inference (requiring T forward passes and a large computational graph for backpropagation), distill the multi-step flow policy into a single-step model:

$$\mathcal{L}_{\text{distill}}(\omega) = \mathbb{E}_{s \sim \mathcal{D}, z \sim \mathcal{N}} \left[\left\| \pi_\omega(z, s) - \pi_\phi^{\text{stop}}(z, s) \right\|^2 \right].$$

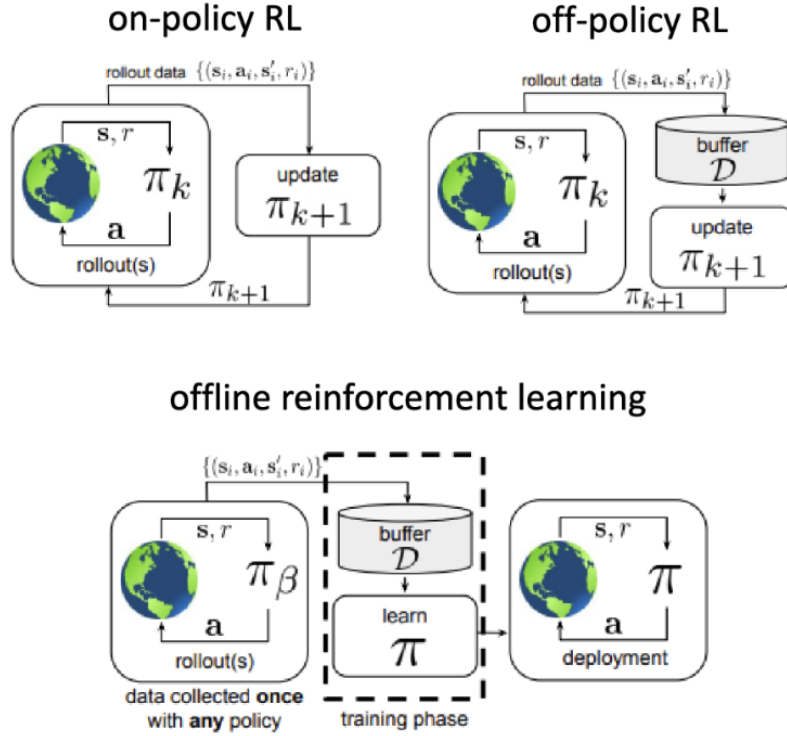


Figure 1: On-policy, off-policy, and offline RL compared (from Sergey Levine CS185 Sp26 slides, as used in Mark’s presentation). In the offline setting, data is collected once under any policy π_β , stored in a buffer, and the policy is learned and deployed without further environment interaction.

The one-step policy π_ω takes noise z and state s directly to an action, without running Euler integration.

4. **Add Q-function loss to distillation:** The final training objective for π_ω combines distillation (policy constraint) with Q-value maximization:

$$\mathcal{L}(\omega) = \mathcal{L}_{\text{distill}}(\omega) - \alpha \cdot \mathbb{E}_{s \sim \mathcal{D}, z \sim \mathcal{N}}[Q_\psi(s, \pi_\omega(z, s))].$$

The hyperparameter α controls the trade-off between staying close to the behavior policy (regularization) and maximizing Q (reward improvement).

Theoretical Caveat: Bias of FQL

The presenter mentioned a “sneak peek” from the adjoint matching perspective. The key insight (attributed to a related theoretical analysis) is:

“FQL replaces the flow policy with a one-step distilled student under a Wasserstein-style regularizer, which solves a different optimization problem whose fixed point is not the tilted distribution.”

The “tilted distribution” here refers to the reward-weighted behavior distribution $\pi^*(a | s) \propto \pi_\beta(a | s) \exp(Q(s, a)/\alpha)$, which is the theoretically correct target. Because FQL uses distillation rather than direct KL regularization, it converges to a biased approximation of this distribution. However, the presenter noted: “This is biased, but pretty good in practice.”

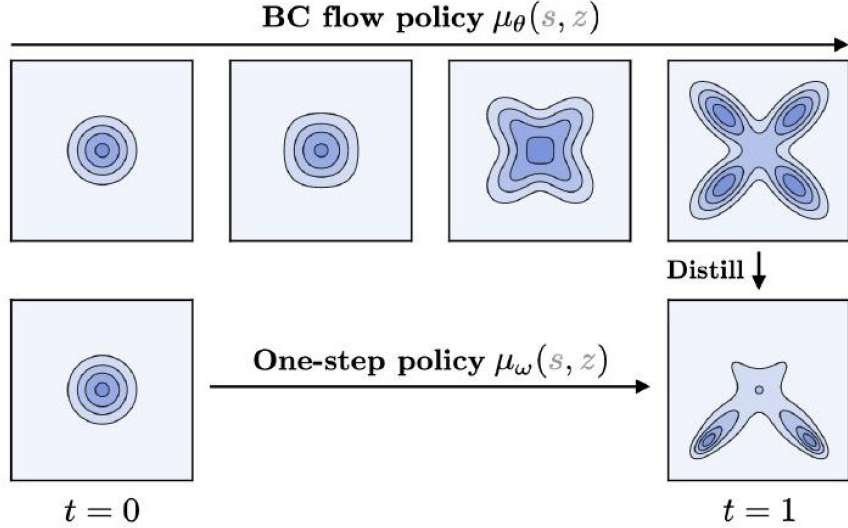


Figure 2: Figure 1 from the FQL paper (slide 12). The BC flow policy $\mu_\theta(s, z)$ evolves a Gaussian over T steps into a complex multimodal distribution. The one-step policy $\mu_\omega(s, z)$ is distilled from it in a single forward pass, then additionally optimized toward high- Q actions.

Experimental Results and Evidence

The presenter pointed to results on standard offline RL benchmarks (referenced at <https://markogata.com/projects/2026/rl-offline/>). FQL performs better compared to less expressive policy representations (e.g., Gaussian-based policies). It was compared against DDPM-based policies (IQL + diffusion), SAC+BC (soft actor-critic with behavioral cloning regularization), and others. The flow-based representation is credited with capturing the multimodality of human demonstration data better than Gaussian baselines. The presenter’s own experience with a robot arm and 30 teleoperation demonstrations was mentioned as illustrative of why multimodal action distributions matter.

Discussion and Q&A

- **Audience asked** about the distinction between on-policy, off-policy, and offline RL. The presenter clarified using a globe/buffer/database analogy from Sergey Levine’s slides: on-policy discards data after each update; off-policy reuses it via a replay buffer; offline RL has a fixed dataset and cannot query the environment.
- **Audience asked** what the first “box” (the world/globe) in the offline RL diagram represents. It represents the world from which trajectories were originally collected (e.g., a human teleoperating a robot).
- **Audience asked** about the multimodal motivation (tree example): why is a Gaussian problematic? The presenter explained that if the dataset shows humans going both left and right around a tree, a Gaussian mean averages to going straight into the tree.
- **Audience asked** whether x_t in the flow objective is the noised action at time t . The presenter confirmed: t is the flow time step and x_t is the interpolated (noised) action.
- **Audience asked** whether the one-step policy π_ω is a special architecture. The presenter said no: it is just an MLP that takes (z, s) as input and outputs an action.
- **Audience observed** that the regularization is not exactly KL but is closer to a Wasserstein

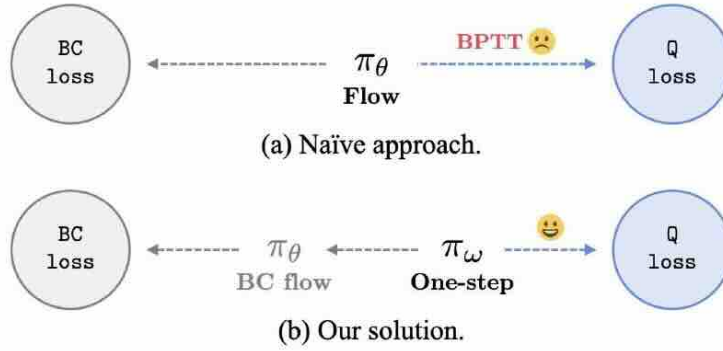


Figure 3: Figure 2 from the FQL paper (slide 13). (a) The naïve approach of directly backpropagating the Q-loss through the multi-step flow policy requires BPTT, which is expensive and unstable. (b) FQL’s solution: the BC flow policy π_θ and the one-step policy π_ω are decoupled; only π_ω receives gradients from the Q-loss.

penalty, and that this is the source of the bias. The presenter agreed and described the adjoint matching perspective as “incredibly dense” math involving Hamiltonian mechanics and ODEs.

- The presenter noted that he skipped a related paper (Adjoint Matching) because it involved “absolutely heinous” math including Hamiltonian mechanics and physics-based field theory.

Key Takeaways and Open Questions

1. Flow models are a natural fit for offline RL because human demonstration data is multimodal; Gaussian policies average over modes and produce poor behavior.
2. FQL’s three-component design (flow BC + Q-function + one-step distillation) is clean and achieves strong empirical results, at the cost of theoretical exactness (biased fixed point).
3. The one-step distillation step is critical for computational tractability: multi-step flow inference creates very large computational graphs incompatible with efficient Q-function gradient backprop.
4. Open questions: (a) Can the bias of FQL be corrected while maintaining computational efficiency? (b) How does FQL scale to real robotics datasets with high-dimensional observations? (c) The presenter’s adjoint matching connection suggests a “correct” alternative exists that is more mathematically grounded — but is it practically competitive?
5. The connection between offline RL policy constraints and optimal transport / Wasserstein distances is a rich theoretical direction that deserves further exploration.

Overview

METRA (Metric-Aware Abstraction for Unsupervised Skill Discovery) is an Oral paper at ICLR 2024 that proposes a method for unsupervised pre-training in reinforcement learning. The motivating observation is that NLP and vision have scaled dramatically by leveraging unsupervised pre-training on vast unlabeled corpora; RL has not achieved analogous scaling, partly because there is no obvious reward-free pre-training objective. METRA addresses this by learning a set of diverse “skills” (a latent-conditioned policy $\pi(a \mid s, z)$) using an intrinsic reward grounded in the **temporal distance metric** — the minimum number of actions required to travel between two states. By maximizing progress in a skill direction z as measured in a latent space that approximates temporal distance, METRA learns skills that explore the most temporally extended directions of the state space, analogously to how PCA finds the directions of maximum Euclidean variance.

Key Ideas and Technical Details

Unsupervised Pre-training for RL

In supervised RL, the data is $\tau = \{(s_1, a_1, s_2, r_1), \dots\}$. In the unsupervised (reward-free) setting, we have $\tau = \{(s_1, a_1, s_2, a_2, \dots)\}$ — no reward signal. The objective is to learn a policy that provides a strong initialization for downstream, reward-supervised fine-tuning.

Skill-Conditioned Policies

Because a single monolithic policy cannot be task-agnostic across all possible downstream tasks, METRA (and the broader unsupervised skill discovery literature) learns a **set** of behaviors. This is formalized by conditioning the policy on a latent “skill vector” $z \in \mathbb{R}^d$:

$$\pi(a \mid s, z), \quad z \sim \mathcal{N}(0, I).$$

Different values of z index different behavioral modes. The goal is for the skills to be (1) **diverse** (conditioning on different z leads to meaningfully different behavior) and (2) **covering** (skills together visit a wide variety of states).

The Problem with Mutual Information

The baseline approach in unsupervised skill discovery (e.g., DIAYN) maximizes the mutual information $I(S; Z)$ between visited states S and skills Z , with various entropy terms:

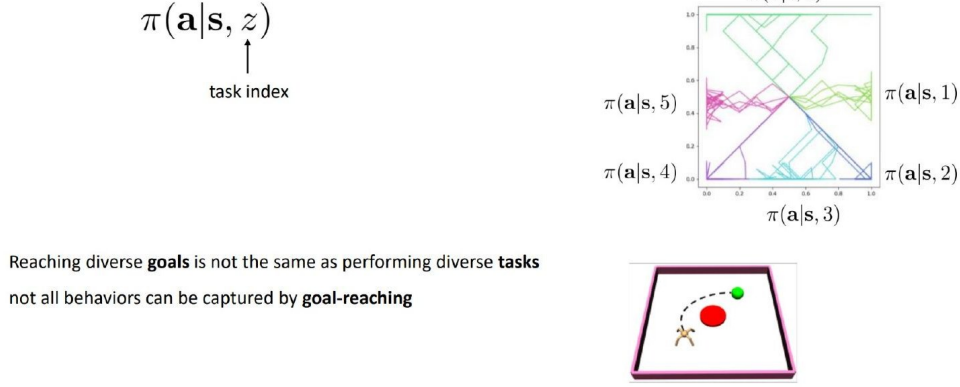
$$r(s, a, z) = I(S; Z) + H(A \mid S) - I(A; Z \mid S).$$

The problem is that mutual information is **invariant to invertible transformations** of the state space — it is “isotropic” in the sense that all dimensions are treated equally. In a high-dimensional state space (e.g., an ant robot in a maze, with 30+ dimensions including x - y position, joint angles, velocities, etc.), this means the skills try to vary uniformly across all dimensions. But varying a joint angle dimension is much less temporally “meaningful” than varying x - y position.

Temporal Distance

The **temporal distance** $d_T(s_1, s_2)$ between two states is defined as the minimum number of actions required for an agent to travel from s_1 to s_2 under the environment dynamics. This is a proper metric (positivity, symmetry, triangle inequality) that is:

Learning diverse skills



Intuition: different **skills** should visit different **state-space regions**

Figure 4: Slide 9 from Deepinder’s METRA presentation. Left: the skill-conditioned policy $\pi(\mathbf{a} \mid \mathbf{s}, z)$ where z acts as a task index. Top right: trajectories of six distinct skills ($z = 0, \dots, 5$) in a simple 2D environment, each covering a different region. Bottom right: an ant locomotion task where a goal state (green dot) is to be reached from the starting position. The key insight is that reaching diverse goals is not the same as performing diverse tasks.

- Invariant to the coordinate representation of states.
- Sensitive to the true reachability structure of the environment.

Note that two states can be close in Euclidean distance yet far in temporal distance (e.g., two sides of a wall in a maze).

METRA’s Core Insight: Metric-Aware Skill Learning

METRA replaces Euclidean variance with **temporal-distance variance**. The intuition: in an ant maze, the x - y position dimensions are highly temporally spread (large changes in position correspond to large temporal distances), while joint-angle dimensions are temporally shallow (joint angles can change in one step). METRA learns skills that maximize coverage of the most temporally spread dimensions.

Formally, METRA learns a mapping $\phi : \mathcal{S} \rightarrow \mathbb{R}^d$ such that Euclidean distance in the latent space approximates temporal distance:

$$\|\phi(s) - \phi(s')\|_2 \approx d_T(s, s').$$

This is analogous to a nonlinear, temporal-distance-aware PCA: instead of finding directions of maximum Euclidean variance, METRA finds the directions in the latent space of maximum temporal-distance variance.

Practical Instantiation

1. Initialize encoder $\phi : \mathcal{S} \rightarrow \mathbb{R}^d$ and policy $\pi(a \mid s, z) : \mathcal{S} \times \mathbb{R}^d \rightarrow \Delta(\mathcal{A})$.
2. Sample a skill direction $z \sim \mathcal{N}(0, I)$.
3. Set the intrinsic reward for transition (s, a, s') :

$$r(s, a, s', z) = (\phi(s') - \phi(s))^\top z.$$

first, we learn a mapping ϕ from **states** to a **latent space** $\mathbf{z}$ such that the **Euclidean distance** in the latent space is equivalent to the **temporal distance** between the states in state

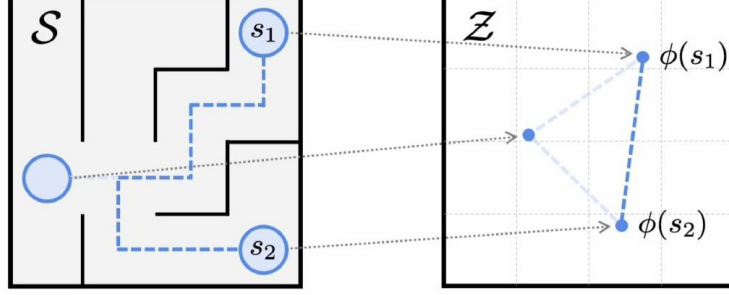


Figure 5: Slide 22: the distance-preserving mapping ϕ . Left: state space $\mathcal{S}$ with a maze; states s_1 and s_2 are far apart in temporal distance (long dashed path through the maze). Right: latent space $\mathcal{Z}$; $\phi(s_1)$ and $\phi(s_2)$ are correspondingly far apart in Euclidean distance. The mapping satisfies $d^*(s_1, s_2) = \|\phi(s_1) - \phi(s_2)\|$.

This measures: how much does the progress from s to s' in the latent space align with the target skill direction z ?

4. Train $\pi(a \mid s, z)$ and $\phi(s)$ jointly to maximize the expected cumulative intrinsic reward, subject to the Lipschitz constraint:

$$\|\phi(s') - \phi(s)\|_2 \leq 1 \quad \text{for all adjacent state pairs } (s, s').$$

This constraint is enforced via dual descent and prevents the trivial solution of ϕ “blowing up” in magnitude.

5. The cumulative reward decomposes as:

$$\sum_t r_t = \sum_t (\phi(s_{t+1}) - \phi(s_t))^\top z = (\phi(s_F) - \phi(s_0))^\top z,$$

a telescoping sum. The policy is thus incentivized to maximize the displacement $\phi(s_F) - \phi(s_0)$ in the direction of z . If ϕ captures temporally spread dimensions (like x - y position), the reward is large; for temporally shallow dimensions (noisy joint angles), the reward is small.

Downstream Fine-tuning

After pre-training, the skill-conditioned policy $\pi(a \mid s, z)$ is frozen. A lightweight “controller” $p(z \mid s)$ is trained via RL on the real reward signal. The controller selects which pre-trained skill to execute; actions are then sampled as $z \sim p(z \mid s)$, $a \sim \pi(a \mid s, z)$. This reduces a complex low-level control problem to a simpler problem of skill selection.

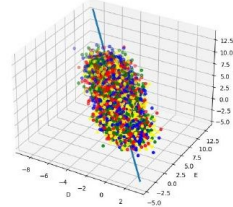
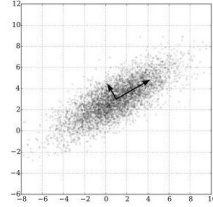
METRA vs. HILP

The presenter noted that the paper is motivated from the perspective of the Wasserstein Dependency Measure, but they chose to present it through the lens of a follow-up paper, HILP (Hilbert Representations), which they found more geometrically intuitive. There are deep connections to RKHS (Reproducing Kernel Hilbert Spaces) and to the concept of a *submetry* (related to quotient spaces) rather than an isometry between the state space and skill space.

METRA as PCA

Illustration on next slide

- **PCA:** learns the directions in a “space” (set of data) that have maximum variance, in Euclidean position.



- **METRA:** learns the directions in a space (the state-space) that have maximum variance, in the temporal distance metric.
- METRA can be seen as a sort of nonlinear PCA in temporal distance space!

Figure 6: Slide 18 from Deepinder’s presentation: METRA as PCA. Left: standard PCA on 2D data finds the direction of maximum Euclidean variance (arrow). Right: METRA applied to a 3D skill space finds the direction of maximum variance under the temporal distance metric. METRA is a nonlinear PCA in temporal-distance space.

Experimental Results and Evidence

The presenter referenced the paper’s website (seohong.me/projects/metra/) for full experimental results. Key claims:

- METRA’s skills are visually interpretable: in an ant maze, skills correspond to navigating in different compass directions, with z controlling the direction of movement in the x - y plane.
- Compared to DIAYN (mutual-information-based skill discovery), METRA produces skills with much better state coverage because it targets the temporally extended dimensions.
- Downstream fine-tuning with the frozen METRA policy and a simple $p(z | s)$ controller outperforms end-to-end fine-tuning baselines in several locomotion and navigation tasks.

Discussion and Q&A

- **Audience asked** what “unsupervised” means in the RL context. The presenter clarified: it means having trajectories without reward labels, i.e., just $(s, a, s', a', \dots)$.
- **Audience asked** if this is the same as goal-conditioned RL. The presenter said it is more general: z is a skill/behavior vector, not necessarily a goal state.
- **Audience asked** about the relationship to orthogonality. The presenter noted that the inner product in the reward function provides a “soft” version of orthogonality: skills are encouraged to be in different directions, with the inner product serving as a continuous rather than binary measure.
- **Audience suggested** using a higher-dimensional embedding or kernel trick to handle non-linear relationships between skills. The presenter acknowledged this is related to the RKHS connection in the paper.
- **Audience asked** whether the temporal distance metric is well-defined when actions are stochastic. The presenter deferred this question, noting it was a good one that would take too

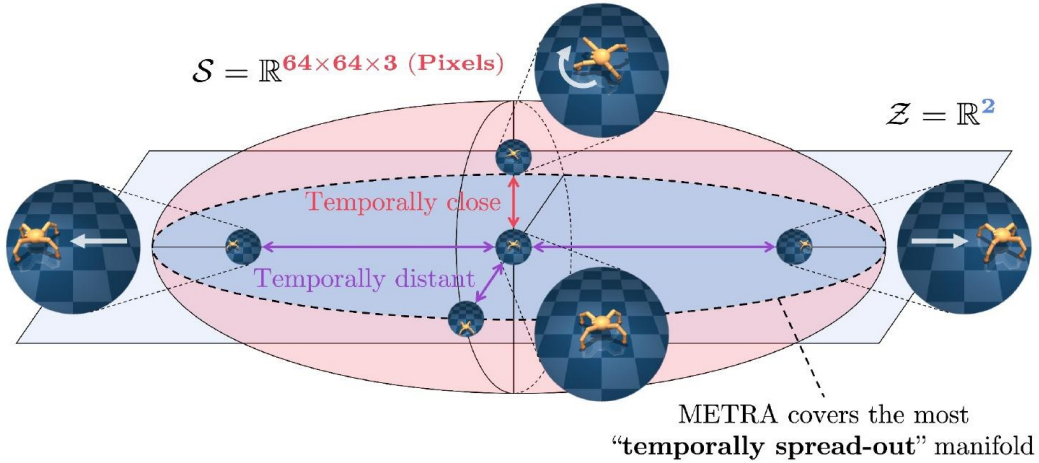


Figure 7: Slide 19 from Deepinder’s presentation. The state space $\mathcal{S} = \mathbb{R}^{64 \times 64 \times 3}$ (pixel observations of an ant) is mapped to a 2D skill space $\mathcal{Z} = \mathbb{R}^2$. The red ellipse represents the temporally spread-out manifold (horizontal axis = x - y locomotion, temporally distant); the blue ellipse represents temporally shallow dimensions (e.g., joint angles, which can change in one step). METRA targets the former.

long to address.

- **Audience asked** whether the temporal distance is really just about the x - y position dimensions. The presenter clarified: it is any dimension that varies a lot in temporal distance, not limited to position (though that is the typical example).
- **Audience noted** that the slide deck mentioned an RKHS connection and submetry, and asked about these. The presenter acknowledged these are real but said they do not fully understand the connection.
- **Audience asked** about post-training. The presenter clarified: in post-training, you freeze $\pi(a | s, z)$ and only learn $p(z | s)$, reducing the problem to choosing which skill to use rather than learning low-level actions.
- **Audience asked** if the skills can be indexed by markdown files (natural language). The presenter noted: yes, you could define the z space as the embedding space of text instructions, giving language-conditioned skills.

Key Takeaways and Open Questions

1. METRA offers a principled way to do unsupervised pre-training in RL by grounding skill diversity in temporal distance rather than Euclidean or mutual-information-based measures.
2. The PCA analogy is powerful: METRA finds the “principal components” of the state space as measured by temporal distance, focusing skill learning on the most navigable/reachable directions.
3. The constraint on ϕ is the key technical ingredient preventing collapse and ensuring ϕ encodes temporal structure.
4. Open questions: (a) How does METRA handle stochastic dynamics where temporal distance is not deterministic? (b) Can the RKHS and submetry connections be made more explicit

and useful? (c) Can the skill space z be given semantic meaning (e.g., via language)? (d) Is there a principled way to determine the dimensionality d of the skill space?

5. The HILP follow-up paper appears to offer a more geometrically mature treatment of the same ideas; reading HILP alongside METRA is recommended.

Overview

This presentation surveyed the field of Randomized Numerical Linear Algebra (RandNLA) and its applications in modern large-scale machine learning. RandNLA replaces deterministic algorithms for linear algebra computations (e.g., exact matrix multiplication, SVD, eigendecomposition) with randomized algorithms based on **random sketching**: compressing a large problem into a much smaller one via a random projection or sampling operator, then solving the smaller problem. The trade-off is probabilistic (rather than worst-case) error bounds in exchange for dramatically reduced runtime and memory. The presentation focused on three application areas — distributed learning, LLM inference quantization, and KV cache compression — and concluded with the Nyströmformer as a concrete application achieving $O(n)$ attention score computation.

Key Ideas and Technical Details

What is Randomized Numerical Linear Algebra?

- **Linear Algebra**: vectors, matrices, vector spaces, eigenvalues/eigenvectors, SVD.
- **Numerical Linear Algebra (NLA)**: algorithms for approximate computations in finite-precision arithmetic; typically provides worst-case error bounds.
- **Randomized NLA (RandNLA)**: uses randomized rather than deterministic algorithms, mostly via **random sketches** that turn a large problem (e.g., $n \times n$ matrix operations) into a much smaller problem ($s \times s$ for $s \ll n$); provides probabilistic error bounds; significantly faster runtimes and lower memory.

Why RandNLA for Machine Learning?

Three main motivating applications were presented:

(1) High Communication Costs in Distributed Deep Learning. In distributed training, the bottleneck is often sending weight updates between a server and client devices. Randomized sketching is *linear* and *unbiased*, making it suitable for compressing model updates before transmission. It is also compatible with secure aggregation for federated learning.

(2) LLM Inference: Post-Training Quantization (PTQ). Quantizing model weights below 4 bits causes performance degradation, largely due to extreme outlier activations. Approaches using RandNLA include:

- *Stochastic rounding*: an unbiased rounding operation.
- *Random sketches/projections for preprocessing activations* (e.g., the HIGGS method), which reduce the influence of outliers before quantization.

(3) LLM Inference: KV Cache Compression. Randomized algorithms (including sketching) can compress the KV cache. A recent example is TurboQuant (March 2026, Google), which achieves extreme KV cache compression ratios.

The AI Memory Wall. The underlying motivation is the “AI memory wall”: memory bandwidth is scaling much more slowly than both model parameter counts and hardware FLOP counts. Since the mid-2010s, transformer parameter counts have grown rapidly while GPU/TPU memory capacity has grown slowly; training FLOP counts have also grown rapidly. This divergence makes memory bandwidth an increasingly critical bottleneck for both training and inference.

Why RandNLA? - AI “Memory Wall”

AI and Memory Wall

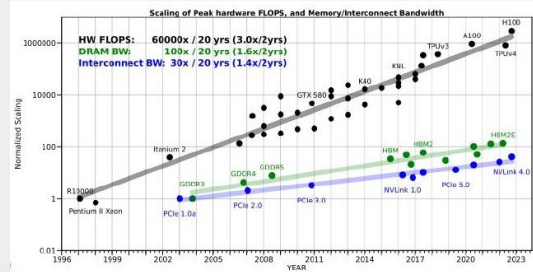
Amir Gholami^{1,2} Zhewei Yao¹ Sehoon Kim¹ Coleman Hooper¹ Michael W. Mahoney^{1,2,3} Kurt Keutzer¹¹University of California, Berkeley ²ICSI ³LBNL

Fig. 1: The scaling of the bandwidth of different generations of interconnections and memory, as well as the Peak FLOPS. As can be seen, the bandwidth is increasing very slowly. We are normalizing hardware peak FLOPS with the R10000 system, as it was used to report the cost of training LeNet-5 [22].

Figure 8: Slide 8 (Surya’s presentation), from Gholami et al. “AI and Memory Wall.” Hardware FLOPS have grown at $3\times$ per 2 years ($60,000\times$ over 20 years), while DRAM bandwidth has grown at only $1.6\times$ per 2 years ($100\times$) and interconnect bandwidth at $1.4\times$ per 2 years. The widening gap makes memory the primary bottleneck for AI training and inference.

Introduction to Randomized Sketching

Consider approximating the matrix product AB where $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$. Writing the product as a sum of outer products:

$$AB = \sum_{j=1}^n A_{:,j} B_{j,:},$$

a natural idea is to sample $s \ll n$ column-row pairs. Define a sketching operator $S \in \mathbb{R}^{s \times n}$ where the t -th row is all zeros except for position j_t , which equals $\frac{1}{\sqrt{s p_{j_t}}}$. Then:

$$AB \approx \frac{1}{s} \sum_{t=1}^s \frac{1}{p_{j_t}} A_{:,j_t} B_{j_t,:}.$$

Choosing probabilities. A theorem states: if $p_i \propto \|A_{:,i}\|_2 \cdot \|B_{i,:}\|_2$ (importance sampling proportional to the product of column and row norms), then the expected Frobenius-norm error is minimized. This makes S an unbiased estimator of AB .

More generally, S is a **subspace embedding**: it approximately preserves the geometry of entire row/column subspaces, not just specific matrices.

The Johnson–Lindenstrauss Lemma

A foundational result in RandNLA: given N points in $\mathbb{R}^n$ (with n very large), there exists a map $f: \mathbb{R}^n \rightarrow \mathbb{R}^k$ with

$$k = O\left(\frac{\log N}{\varepsilon^2}\right),$$

such that for all pairs of points u, v :

$$(1 - \varepsilon)\|u - v\|_2^2 \leq \|f(u) - f(v)\|_2^2 \leq (1 + \varepsilon)\|u - v\|_2^2.$$

Why RandNLA? - AI “Memory Wall”

AI and Memory Wall

Amir Gholami^{1,2} Zhewei Yao¹ Sehoon Kim¹ Coleman Hooper¹ Michael W. Mahoney^{1,2,3} Kurt Keutzer¹

¹University of California, Berkeley ²ICSI ³LBNI

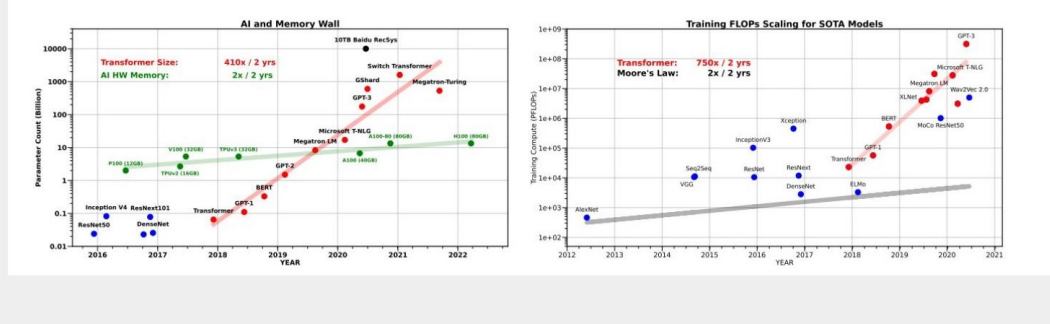


Figure 9: Slide 9 (Surya’s presentation). Left: transformer model sizes (parameter counts, blue dots) have grown at $410\times/2$ years, while AI hardware memory (green dots) has grown at only $2\times/2$ years — a rapidly diverging gap. Right: training FLOPs for SOTA models have grown at $750\times/2$ years, far outpacing Moore’s Law ($2\times/2$ years).

This shows that pairwise distances can be preserved to within an ε factor using a projection to only $O(\log N/\varepsilon^2)$ dimensions — a dramatic dimensionality reduction.

Application: Nyströmformer (2021)

Standard attention computation requires $O(n^2)$ operations (for sequence length n) due to the softmax over all pairwise similarities $QK^\top$.

The Nyström method for matrix approximation: given a matrix $S \in \mathbb{R}^{n \times n}$ (e.g., the softmax attention matrix), choose $m \ll n$ landmark rows/columns to form an $m \times m$ submatrix $\hat{S}$. The Nyström approximation of S can be constructed from $\hat{S}$, F_S (“front” block), and B_S (“back” block) in $O(nm^2)$ time.

Nyströmformer does not directly apply the Nyström method (since computing any single element of the softmax attention matrix requires $O(n)$ work for normalization), but uses a **clever approximation** that achieves the same $O(nm)$ complexity, giving overall $O(n)$ attention (for fixed m).

Compared to FlashAttention: Nyströmformer is a *algorithmic* approximation (hardware-agnostic), whereas FlashAttention is a *systems* optimization (specific CUDA memory-access patterns) that achieves the exact attention computation in less wall-clock time. Both can in principle be combined.

Experimental Results and Evidence

The presenter cited the Nyströmformer paper’s results showing that the approximate attention scores achieve competitive performance on standard NLP benchmarks at a fraction of the computational cost of full attention. Specifically, the method runs in $O(nm^2)$ with $m \ll n$, effectively achieving $O(n)$ for fixed m .

For distributed learning and quantization, the presenter referenced HIGGS and TurboQuant as concrete examples but did not present specific benchmark numbers in detail.

Discussion and Q&A

- **Audience member expressed skepticism** about the $O(n)$ attention result: “I find that result incredible. What’s the loss there, like what’s the error bound?” The presenter acknowledged the bound exists and is “pretty good” but deferred full details.
- **Audience asked** why RandNLA has not been more widely adopted, given the apparent benefits. The presenter noted two reasons: (1) limited awareness, and (2) insufficient engineering investment — randomized algorithms are still primarily developed in the numerical algorithms community rather than ML engineering teams. However, the presenter noted that looking into implementation details of tools like TurboQuant reveals randomized components.
- **Audience asked** whether deployed systems (Gemini, ChatGPT) use RandNLA. The presenter said probably yes in some form, at least for serving (stochastic rounding, KV cache compression), but this is often not prominently advertised.
- **Audience asked** about quantization below 4 bits and why outlier activations are the problem. The presenter explained: quantization maps a float32 value to a small integer, and if there are extreme outliers, the quantization grid is forced to be very wide, making it coarse for typical values and leading to precision loss.
- **Audience asked** whether Nyströmformer and FlashAttention can be combined. The presenter said possibly: they address different aspects of the attention computation (algorithmic approximation vs. hardware efficiency), so combining them is plausible.
- **Audience asked** about the Johnson–Lindenstrauss lemma’s choice of $k = O(\log N/\epsilon^2)$ and whether it seems “random.” The presenter confirmed: “Just assume k is $O(\log N/\epsilon^2)$. That’s all that matters.”
- **Audience asked** whether sampling in the sketching algorithm is with replacement. The presenter confirmed yes.

Key Takeaways and Open Questions

1. RandNLA offers a principled framework for trading worst-case guarantees for probabilistic ones in exchange for significant runtime and memory savings — highly relevant as the AI memory wall becomes increasingly limiting.
2. The Johnson–Lindenstrauss lemma provides the theoretical foundation: high-dimensional geometry can be approximately preserved with logarithmic projection dimension.
3. Nyströmformer demonstrates that $O(n^2)$ attention can be approximated in $O(n)$ time via sketching, with competitive accuracy — a striking result that the audience found almost “too good to be true.”
4. Open questions: (a) Can Nyströmformer and FlashAttention be combined to get both algorithmic and systems efficiency? (b) How do randomized quantization methods (HIGGS, stochastic rounding) compare to deterministic methods at scale? (c) What are the best practices for choosing sketching operators in the context of LLM fine-tuning vs. inference?
5. The field of RandNLA is underexplored in the ML community relative to its potential impact; crossover between numerical analysis and ML engineering communities could yield significant practical gains.

Overview

The Muon optimizer, introduced by Keller Jordan (researcher at OpenAI), is a novel gradient-based optimization algorithm for neural network training that replaces the standard gradient update with an **orthogonalized** gradient update. The core idea is to apply Newton–Schulz iterations to the momentum buffer of the gradient, transforming its singular values toward 1 before using it as the weight update. This produces a gradient update matrix with a low condition number — promoting more uniform exploration across parameter space — in contrast to Adam/AdamW, which empirically results in high-condition-number updates that favor certain directions. Muon has achieved record performance on the NanoGPT speedrun benchmark and has been adopted (with modifications) in Moonshot AI’s Kimi series of large language models (including Kimi K2, a 1.6-trillion-parameter model), marking a significant transition from a toy algorithm to production-scale LLM pre-training.

Key Ideas and Technical Details

Motivation: Desiderata for an Optimizer

The presenter motivated the design of Muon around three desiderata for an optimizer:

1. **Fast convergence:** converge to a good solution in fewer steps or less wall time.
2. **Good solutions:** avoid suboptimal local minima; encourage exploration of parameter space.
3. **Robustness:** not overly sensitive to hyperparameter choices.

Standard SGD achieves convergence but can be slow; Adam adds momentum and adaptive learning rates; Shampoo adds a full second-order (approximate Hessian) preconditioning. Muon takes a different approach: rather than adapting learning rates per parameter, it **orthogonalizes** the gradient update matrix.

The Muon Algorithm (Simplified)

Step 1 — Compute gradient:

$$G_t = \nabla_W \mathcal{L}(W_{t-1}).$$

Step 2 — Momentum update:

$$B_t = \mu \cdot B_{t-1} + G_t,$$

where μ is the momentum coefficient and B_t is the momentum buffer (running average of gradients). (B stands for buffer, not weights.)

Step 3 — Newton–Schulz orthogonalization:

$$O_t = \text{NewtonSchulz}(B_t).$$

This maps B_t to a matrix with singular values close to 1 (detailed below).

Step 4 — Weight update:

$$W_t = W_{t-1} - \eta \cdot O_t,$$

where η is the learning rate.

Newton–Schulz Iteration

Newton–Schulz is an iterative algorithm that maps a matrix to one with singular values close to 1. The key insight is that if you apply an **odd-degree polynomial** p to a matrix $B = U\Sigma V^\top$ (using the SVD), the result is $U p(\Sigma) V^\top$ — the polynomial acts only on the singular values, leaving the singular vectors unchanged. This is the property Muon exploits.

Specifically, Muon uses a **quintic polynomial**:

$$p(t) = at^5 + bt^3 + ct,$$

for coefficients a, b, c chosen so that iterating $t \leftarrow p(t)$ converges to 1 for $t \in (0, 1]$. The polynomial is applied iteratively:

$$B^{(k+1)} = p(B^{(k)}) = a(B^{(k)})^5 + b(B^{(k)})^3 + cB^{(k)},$$

where matrix powers are computed via repeated matrix multiplication. After enough iterations, the singular values of $B^{(k)}$ all approach 1. In practice, only a small number of iterations (e.g., 5) are needed due to the quintic convergence rate.

Why odd degree? An odd polynomial $p(\sigma)$ ensures $p(-\sigma) = -p(\sigma)$, which preserves the sign structure of singular values (preventing spurious sign flips in the update).

Why quintic? Quintic gives faster convergence than cubic (fewer iterations needed) while remaining computationally tractable.

Effect: Lowering the Condition Number

The **condition number** of a matrix is $\kappa = \sigma_{\max}/\sigma_{\min}$ (ratio of largest to smallest singular value). A high condition number means updates are dominated by a small number of directions (the large singular-value directions), while other directions are barely updated.

Empirically, Adam and AdamW produce weight update matrices with **high condition numbers**: updates favor certain directions and are weak in others. Muon, by mapping all singular values to ≈ 1 , produces a condition number near 1, meaning all directions are updated equally. This is an inductive bias toward more uniform exploration of parameter space.

Moonshot AI’s Kimi Modifications

Moonshot AI adapted Muon for production-scale training of Kimi K2.6 (then K2, 1.6T parameters) with several modifications:

- **Muon only for hidden (weight matrix) layers:** AdamW is used for embedding layers, normalization layers, and other non-matrix layers. Only the hidden-state projection matrices (e.g., MLP weight matrices) use Muon.
- **RMS scaling:** The update is scaled by a factor related to the matrix dimensions (specifically $\sim 1/\sqrt{\max(\text{fan_in}, \text{fan_out})}$, [exact form unclear]), which improves training stability. A theorem in the Moonshot paper justifies this specific scaling.
- **Distributed version:** For trillion-parameter models, a distributed implementation of Newton–Schulz was necessary. Since all Newton–Schulz operations are matrix multiplications, they can be parallelized and implemented as efficient CUDA kernels; hardware providers are reportedly writing optimized kernels.

Mixed optimizer scheme: Muon (for hidden layers) + AdamW (for others). Empirically, this combination works well, contrary to the intuition that mixing optimizers is inadvisable.

Empirical finding on switching optimizers: If a model is pre-trained with AdamW and then post-trained (e.g., for RL fine-tuning) with Muon, performance degrades relative to using the same optimizer for both stages. The mechanism for this is not fully understood.

Experimental Results and Evidence

- **NanoGPT speedrun:** Muon holds the record for fastest training of a small language model to an accuracy threshold on a web benchmark (the specific dataset was not named by the presenter). This was established around 2022–2023.
- **Keller Jordan’s largest model:** approximately 1.5B parameters.
- **Moonshot AI:** trains models up to 1.6T parameters with Muon. At training scales of 16B, 32B, and larger parameters, they observe in practice that AdamW produces higher condition-number updates than Muon, confirming the theoretical motivation.
- The presenter pointed to an interactive visualizer (accessed via the presenter’s website) that allows users to initialize a matrix with a given condition number, apply Newton–Schulz steps, and observe the singular values converge to 1.

Discussion and Q&A

- **Audience observed** that Muon sounds like “Shampoo on steroids.” The presenter noted that Keller Jordan explicitly argues in his blog post that Muon is *not* Shampoo, but acknowledged the similarity; the key difference is that Muon uses the Newton–Schulz orthogonalization while Shampoo uses a different (Kronecker-product-based) preconditioning.
- **Audience asked** why we want to lower the condition number. The presenter explained: a lower condition number means the gradient update does not over-favor certain directions; with condition number ≈ 1 , all directions are updated equally, acting as an inductive bias for broader exploration of parameter space.
- **Audience asked about pre-processing data** to lower the condition number (analogous to whitening inputs). The presenter acknowledged the analogy but noted that in the context of LLM training the relevant conditioning is on the weight-update matrices, not the input data.
- **Audience asked** what the dimensions of B_t are. The presenter noted: for a hidden layer of shape 4096×4096 , B_t is 4096×4096 .
- **Audience asked** how expensive Newton–Schulz is. The presenter noted: all steps are matrix multiplications, which can be implemented as efficient CUDA kernels and run quickly. Hardware vendors are actively writing optimized kernels.
- **Audience asked** whether the mixed optimizer (Muon + AdamW) is advisable. The presenter said that while mixing is typically frowned upon, they find in practice (and Moonshot AI finds at scale) that it works well; different layers may benefit from different update rules, especially when using history-tracking optimizers like Adam.
- **Audience asked** why switching from pre-training optimizer to post-training optimizer (e.g., pre-train with AdamW, post-train with Muon) degrades performance. The presenter noted there is no good explanation yet: “They don’t have an explanation. They just like a measurable difference.” Speculation: the model may become “fitted” to the optimizer’s update statistics in a way that makes switching harmful (“fitting to the optimizer rather than fitting to the task”).
- **Audience member** noted that the idea of exploring more directions is reminiscent of the Neural Thickets paper: both are about encouraging exploration over diverse directions in

parameter space.

- The presenter mentioned wanting to extend the visualizer to higher dimensions and solicited feedback.

Key Takeaways and Open Questions

1. Muon’s key insight is simple and elegant: replace the gradient update matrix with one whose singular values are all close to 1, forcing equal update magnitudes across all parameter directions and lowering the condition number of weight updates.
2. The Newton–Schulz iteration is an efficient way to achieve near-orthogonalization using only matrix multiplications (no explicit SVD), making it practical at scale.
3. Muon has graduated from a toy algorithm (NanoGPT speedrun) to production-scale LLM pre-training (Kimi K2, 1.6T parameters), a remarkable trajectory.
4. The finding that optimizer switching (pre-training to post-training) degrades performance is practically important and theoretically unexplained. Open question: is this specific to Muon, or a general phenomenon with any optimizer switch?
5. Open questions: (a) Is there a theoretical understanding of why equal singular values (condition number ≈ 1) leads to better final model quality, beyond the heuristic of “more uniform exploration”? (b) Can Muon be extended to all layers (not just hidden matrices) without performance loss? (c) Does the optimal momentum parameter μ for Muon differ from that for Adam?